bstage gateway

bstage Gateway 아키텍처 개요

---

1. 서비스 구성

bstage Gateway는 3개의 독립 서비스로 구성됩니다.

서비스	역할
Partner Service	API Key 생성·수정·삭제 관리, 변경 신호 Kafka 전파
bstage-gateway	클라이언트 요청을 Backend로 라우팅, API Key 인증, API 문서 생성
bstage-gateway-mcp	게이트웨이 API를 5분마다 조회하여 AI 도구(MCP 프로토콜)에 제공

---

2. bstage-gateway 내부 구성

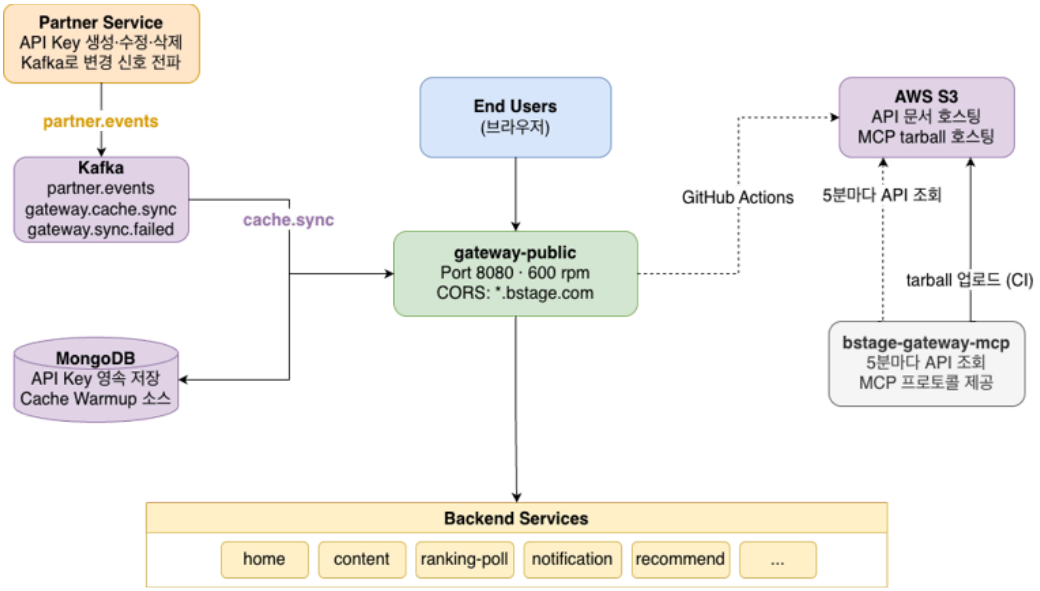
gateway-public, gateway-partner, gateway-common 3개 모듈로 구성됩니다.

모듈	대상	비고
gateway-public	End User (브라우저)	
gateway-partner	Partner Server (S2S)	TBD. public 먼저 진행 후 니즈 생기면 인증 단계부터 추가 설계 예정
gateway-common	공통 모듈	

gateway-common은 두 게이트웨이가 공유하는 인증, 캐시, Kafka 이벤트 소비 로직을 포함합니다.

---

3. 서비스 간 연결 관계



bstage-gateway

- Kafka 토픽 `partner.events`으로 API Key 변경 신호 전파
- bstage-gateway는 신호를 수신해 Partner API를 호출, MongoDB에 저장하고 캐시를 갱신

bstage-gateway → Backend Services

- 인증 완료 후 요청을 해당 Backend로 프록시
- URL 패턴: `http://{service}.{tier}` (예: `http://home.thor`)
- `X-BMF-TIER` 헤더로 tier 값 전달

bstage-gateway → AWS S3

- GitHub Actions(수동 트리거)로 API 문서를 생성하여 S3에 업로드
- 경로: `s3://{ENV_DOCS_BUCKET}/gateway-api-docs/{gateway-type}/`
- dev / qa 환경 전용

bstage-gateway-mcp → AWS S3

- GitHub CI 빌드 시 tarball(.tgz)을 패키징하여 API 문서와 동일한 S3 버킷에 업로드
- 경로: `s3://{DOCS_BUCKET}/mcp/bstage-gateway-mcp.tgz`
- 게이트웨이 문서 index 페이지에서 다운로드 링크 제공

---

4. 사용 인프라

인프라	용도	비고

MongoDB	API Key 영속 저장, 시작 시 Warmup 소스	
Kafka	API Key 변경 이벤트 전파 및 캐시 동기화	
Caffeine	인증용 In-Memory 로컬 캐시	인스턴스별 독립, Redis 미사용
AWS S3	API 문서 + MCP tarball 호스팅	GitHub Actions 경유
Resilience4j	Circuit Breaker (Backend 장애 격리)	실패율 50% → Open, 30초 후 복구
Bucket4j	Rate Limiting	

Redis 는 Rate limit 기능 정식 추가 시 추가 예정

---

5. 인증 처리

두 게이트웨이 모두 동일한 **API Key 인증** 사용.

요청 시 필수 헤더 3개:

`X-BSTAGE-APP-ID` — 공개 식별자

`X-BSTAGE-APP-SECRET` — 비밀키

`X-BSTAGE-TENANT-ID` — 접근할 Space ID

인증 파이프라인 (7단계)

단계	내용
1. 헤더 추출	필수 헤더 3개 존재 확인
2. APP-ID 확인	APP-ID 이상 여부 체크
3. 캐시 조회	Caffeine 캐시에서 API Key 조회, 상태(만료/취소) 확인
4. Secret 검증	SHA-256 해시, 비교
5. Tenant 검증	spaceIds 기반 접근 권한 확인

6. 추가 검증	PUBLIC 타입: Origin 검증 / SECRET 타입: IP Whitelist 검증
7. 컨텍스트 설정	인증 헤더 제거, 내부 헤더(`X-BMF-TIER` , `X-BMF-SID`) 추가

전 단계에서 Local Cache만 사용 → DB/Redis 호출 없음 → **100µs 이하** 처리

---

6. API Key 동기화

Kafka Partition 1개로 자동 Leader 선출. Redis Lock 불필요.

흐름 요약

1. Partner Service가 `partner.events` 발행
2. Leader Pod(Partition 할당된 1개)가 수신 → Partner API 호출 → MongoDB 저장 → `gateway.cache.sync` 발행
3. 모든 Pod가 `gateway.cache.sync` 수신 → MongoDB 조회 → Caffeine 캐시 갱신

Kafka 토픽

토픽	Consumer Group	역할
`partner.events`	`gateway-leader`	Leader Pod만 수신 (Partition 1개)
`gateway.cache.sync`	`gateway-pod-{podId}`	모든 Pod 개별 수신
`gateway.sync.failed`	`gateway-retry`	실패 건 재처리

시작 시 MongoDB에서 전체 로드(Cache Warmup) 후 서비스 시작 → Cold Start 없음

실패건 재처리 시 exponantial backoff + jitter 적용

---

7. API 문서 생성

`openapi.routes` 설정이 라우팅과 문서 생성의 단일 소스입니다.

생성 → 배포 흐름

1. Gradle Task (`./gradlew :api-docs:generateApiSpecs`) 실행

- Backend 서비스들의 `/v3/api-docs` 수집
- 경로 변환(Backend 경로 → 게이트웨이 경로), 내부 헤더 제거, 인증 헤더 정의 추가
- HTML(Scalar UI), JSON(OpenAPI Spec), index.html 생성
- GitHub Actions가 S3에 업로드

dev / qa 환경에서만 활성화됩니다.

---

8. 현재 라우트 목록

gateway-public

서비스	게이트웨이 경로	백엔드
home	/api/home/**	`http://home.{tier}`
content	/api/content/**	`http://content.{tier}`
ranking-poll	/api/ranking-poll/**	`http://ranking-poll.{tier}`
notification	/api/notification/**	`http://notification.{tier}`
recommend	/api/recommend/v1/rankings/board-contents	`http://data-service-lab-api.{tier}`
ads	/api/v2/ads	`http://portal-api.bstage-system`

ads 는 bstage-system 연결, 예외적용 테스트 용으로 게이트 웨이 경로가 공통규칙에 위반됩니다

- **라우팅 공통 규칙**: /api/서비스명/버전/** (버전 부터 backend 서비스의 path 사용)

gateway-partner

서비스	게이트웨이 경로	백엔드

home	/api/partner/home/**	`http://home.{tier}`
content	/api/partner/content/**	`http://content.{tier}`